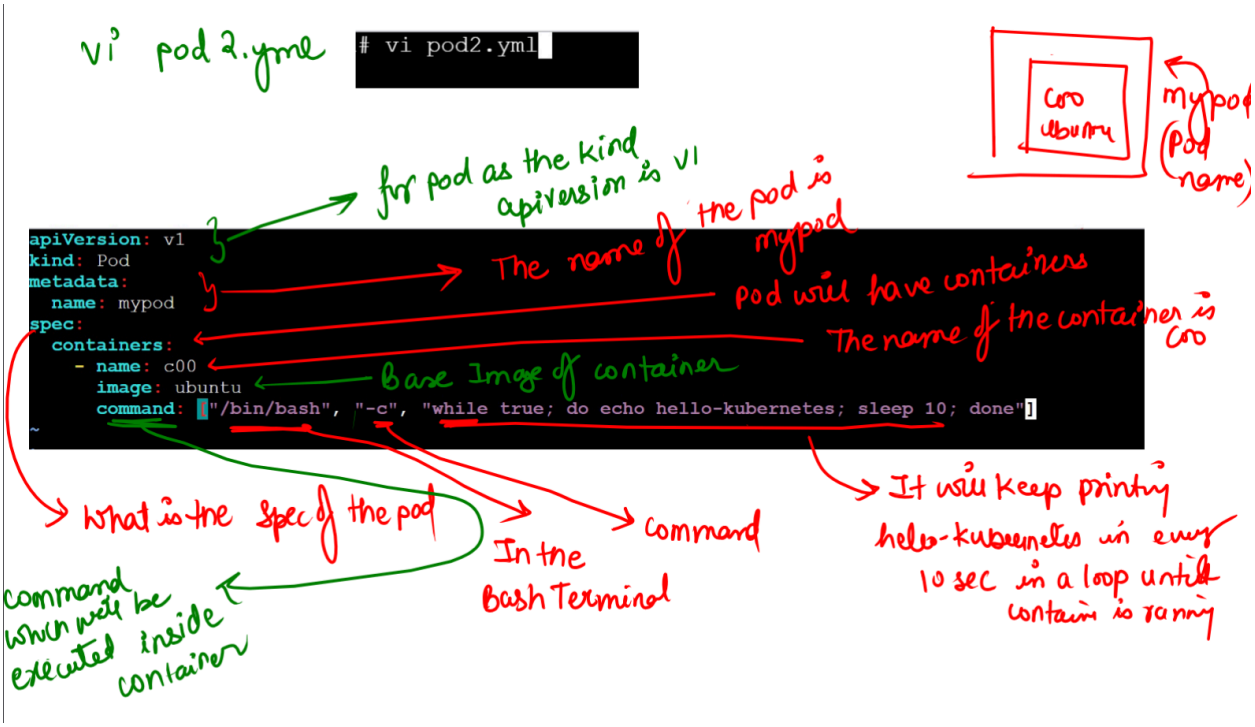
3_Pod

Created by	mp-30
Category	Notes Revision
Last edited by	mp-30
Topics	

Basic pod yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: mypod
spec:
    containers:
        - name: c00
          image: ubuntu
          command: ["/bin/bash", "-c", "while true; do echo
hello-kubernetes; sleep 10; done"]
```



The Core confusion

Most beginners make this mistake:

❌ *Wrong thinking: "Pod = Container, so I can just stuff everything in one Pod"*

✅ *Right thinking: "Pod = Smallest deployable unit that wraps tightly-coupled containers that MUST live together"*

A Pod and a container are NOT the same thing. A Pod is a wrapper - it can hold one container (most common) or multiple containers that are tightly linked.

Why can’t we put 100 Containers in One pod?

Use the rommate logic: Woult you put 100 people in a 2 BHK flat?

Problem 1 — Restart Cascase

- A pod restarts as a single unit
- If your DB container crashes inside a 100-container Pod → your frontend also restarts

- Your frontend did nothing wrong, but it still gets killed.

> The whole flat gets evacuated because one roommate burnt the toast

Problem 2 — Scaling Nightmare

- You want to scale your web-app → Kubernetes scales the entire Pod
- Now you are spinning up 100 containers just to get more web server
- You are wasting memory, CPU, and money on conatainer you did not need to scale.

> You want one more developer, but HR says you must hire 100 people at once.

Problem 3 — Single Responsibility Principle

- Each Pod should do one job well
- 100 containers = 100 responsibilities = debugging nightmare
- When something breaks, you cannot isolate the cause

When do multiple containers GO in one Pod?

There is a valid reason to put 2-3 containers together: when they are so tightly coupled they must share network or storage and must live or die together. This is called the Sidecar pattern.

The Sidecar Example

Your main app writes logs to a file. A log-shipper container reads that file and sends it to Elastic search. These two share a volume and must always run together — so they go in the SAME Pod. Your database? Totally separate Pod.

Scenario	Same Pod?	Reason
App + Log Shipper (Sidecar)	Yes	Shares volumes, tightly coupled
App + Auth Proxy (Ambassador)	Sometimes	Share network, acts as gateway

App + Database	No	Independent lifecycles
App + Cache (Redis)	No	Scale Independently
Frontend + Backend	No	Separate jobs, separate scaling

The 3-Question Test

Before putting containers in the same Pod, always ask these three questions:

#	Question	If yes
1	Do they HAVE to share the same network/ip?	Candidate for same Pod
2	Do they HAVE to share the same storage volume?	Candidate for same Pod3
3	If one crashes, should the other restart too?	Candidate for Same pod

> RULE: If all 3 answers are YES → Same Pod. Even ONE answer is NO → Seperate PODS.

Quick Revision → Remember This

What is a pod?	**What it is NOT**
Smallest deployable unit in Kubenetes	NOT just a single container
A wrapper around 1 (or rarely 2-3) containers	NOT a box to fill with everything
Containers inside share IP and storage	NOT isolated from each other
Restarts as a whole unit	NOT able to restart one container independently
Scheduled together on one Node	NOT spread acros multiple Nodes

One-line Definition to Remember

> A pod is not a box you fill — it is a contract between containers that says: we are so dependent on each other that we must always on the same machine,

> share the same network, and live or die together.